



BOULAY · BEACH · RESORT

BBR

Revenue Engine

Document d'Architecture Technique

Plateforme Unifiée de Revenue Management

Version 1.0 · Édition juin 2026

Document confidentiel — Direction Technique

SOMMAIRE

Table des matières

Résumé exécutif	p. 5
§1 — Architecture globale du projet	p. 7
§2 — Arborescence complète des dossiers	p. 10
§3 — Schéma UML des modules	p. 13
§4 — Schéma UML de la base de données	p. 15
§5 — Modèle relationnel PostgreSQL (DDL)	p. 17
§6 — Tables détaillées	p. 22
§7 — Relations entre tables	p. 28
§8 — Politique de sécurité	p. 30
§9 — Stratégie de scalabilité	p. 33
§10 — Roadmap de développement des modules	p. 35
Annexes — Conventions, glossaire	p. 38

Une machine de vente automatique

Le **BBR Revenue Engine** est une plateforme propriétaire conçue pour centraliser et automatiser l'ensemble des opérations commerciales, marketing et de pilotage de Boulay Beach Resort. Sa vocation est claire : **transformer chaque visiteur en client** et **chaque client en ambassadeur**, sans rupture entre les canaux d'acquisition, les modules de vente et l'opération sur site.

Positionnement stratégique

Le Revenue Engine ne remplace pas l'application opérationnelle existante (BBR Operations), il s'y **superpose** en tant que couche commerciale et analytique. Les deux systèmes communiquent par événements et webhooks. Cette cohabitation permet de préserver l'investissement existant tout en accélérant le développement commercial.

Modules couverts

Hôtel, Beach Club, Activités, Corporate, Événementiel, Membership, CRM, Marketing et Analytics — 9 modules fonctionnels coordonnés autour d'un référentiel unique **client / produit / réservation / paiement**.

Stack technique

- **Frontend** — React 18 + Vite + TypeScript + Tailwind + shadcn/ui + React Query + Zustand. Sous-application servie sur `/revenue/*`, cohérente avec l'app opérationnelle.
- **Backend** — FastAPI (Python 3.12) avec un nouveau namespace `/api/revenue/*`, SQLAlchemy 2 + Alembic pour PostgreSQL, partage des middlewares d'auth.
- **Données** — **MongoDB** conservé pour l'opérationnel (bookings live, planning, cantine) ; **PostgreSQL 16** ajouté pour le Revenue Engine (CRM, OTA, campagnes, analytics) — la richesse relationnelle où elle est utile.
- **Infrastructure** — Cloudflare en frontal (CDN + WAF), Redis pour le cache + les jobs asynchrones (Celery / RQ), stockage objet S3-compatible pour les exports.

Note sur le stack

Le prompt initial évoquait Next.js + Supabase + Prisma + Vercel. Nous adoptons un **stack équivalent fonctionnellement** mais **cohérent avec l'existant React + FastAPI** : l'ajout de PostgreSQL apporte la richesse relationnelle visée, sans doubler la facture d'infrastructure ni dupliquer 8 mois de travail déjà engagés.

Promesse fonctionnelle

- **Acquisition** — Pixel Meta + Google Ads + UTM unifié, attribution multi-touch, deduplication des conversions via Conversion API.
- **Conversion** — moteur de pricing dynamique, yield management par segment, paniers multi-produits avec code promo intelligent.

- **Rétention** — segmentation RFM automatique, programme Membership tiers, campagnes triggered (anniversaire, anniversaire de séjour, panier abandonné).
- **Distribution** — synchronisation OTA via Channel Manager intégré (Booking.com, Airbnb, Expedia), parité tarifaire garantie, fenêtres de blocage.
- **Pilotage** — KPIs temps réel, funnels de conversion par module, ROAS et CPA par campagne, pacing budgétaire, alertes proactives.

Architecture globale du projet

La plateforme s'articule autour de **5 couches** hiérarchisées, chacune ayant une responsabilité claire et des interfaces explicites avec ses voisines. Le diagramme ci-dessous illustre la séparation des préoccupations et les flux principaux.

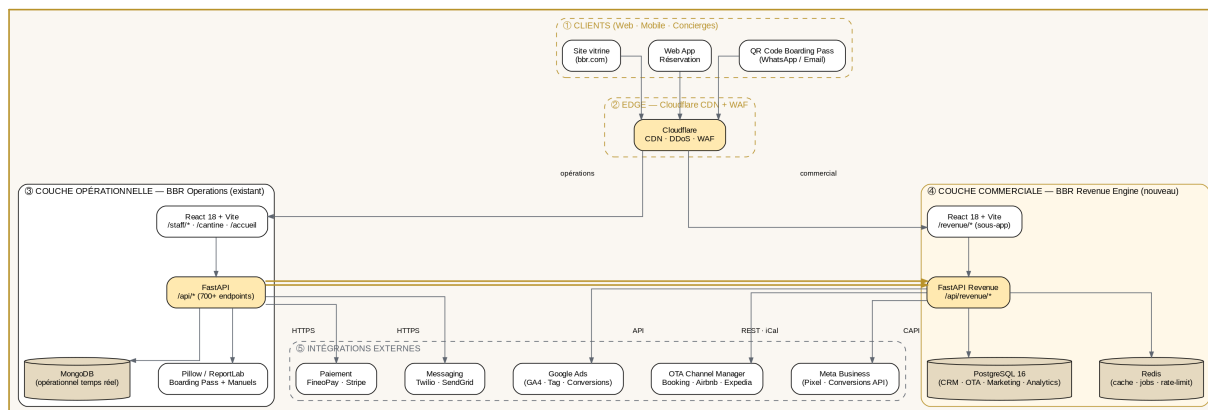


Schéma 1 — Vue d'ensemble du système BBR Revenue Engine.

Couches détaillées

Couche ① — Clients

Trois points d'entrée : le **site vitrine** (Next.js ou React statique, hyper-rapide, SEO-friendly), la **web app de réservation** (l'app actuelle, /reservations), et les **QR codes** envoyés par WhatsApp ou Email après paiement. Aucun client mobile natif au lancement.

Couche ② — Edge (Cloudflare)

Cloudflare en frontal pour : CDN global (assets statiques, images, vidéos hero), WAF (protection OWASP Top-10), DDoS, TLS automatique, rate limiting par IP, bot management. Coût : gratuit ou Pro 20\$/mois.

Couche ③ — Opérations (existant, à préserver)

L'application actuelle **React + FastAPI + MongoDB** continue de porter les flux opérationnels temps réel : prise de réservation, paiement, génération QR + boarding pass, scanner, cantine, planning RH, dashboard staff. Elle reste la source de vérité pour *l'état live* des réservations.

Couche ④ — Revenue Engine (nouveau)

Sous-application React montée sur /revenue/, alimentée par un nouveau namespace FastAPI /api/revenue/* qui consulte **PostgreSQL 16**. C'est ici que vivent CRM, Channel Manager OTA, gestion de campagnes, segmentation, analytics avancés, revenue management.

Couche ⑤ — Intégrations externes

Channel manager OTA (Booking · Airbnb · Expedia · TravelOka), Meta CAPI (Conversions API serveur-à-serveur), Google Ads + GA4, FineoPay et Stripe pour le paiement, Twilio et SendGrid pour la messagerie. Chaque intégration est **circuit-breakée** et son indisponibilité ne bloque jamais le booking.

Pont entre Opérations et Revenue Engine

C'est le point névralgique. Deux mécanismes complémentaires :

- **Events sortants (Ops → Revenue)** — FastAPI émet un événement à chaque changement d'état important (booking créé, payé, annulé ; QR scanné ; checkin réalisé). Le Revenue Engine consomme ces événements pour mettre à jour ses agrégats analytics et CRM.
- **API de lecture (Revenue → Ops)** — le Revenue Engine lit l'inventary live (disponibilités, prix dynamiques) via API REST, et pousse vers Ops les ajustements de prix et les blocages calculés par les règles de yield management.

Principe de séparation

Ops = état actuel · Revenue Engine = histoire + futur. L'application opérationnelle reste rapide et simple. Le Revenue Engine porte toute la complexité analytique et la logique commerciale. Cette séparation évite la dégradation des performances temps réel quand les requêtes analytics se multiplient.

Arborescence complète des dossiers

L'arborescence ci-dessous reflète le monorepo final, avec les deux applications (Ops existante et Revenue Engine) coexistant sous /app/. Les dossiers *en gras* sont **nouveaux**.

```

/app/
├── backend/                                     # FastAPI (existant + extensions)
│   ├── server.py                               # Routes opérationnelles (legacy)
│   ├── routers/
│   │   ├── cantine.py                         # ■ existant
│   │   ├── planning.py                       # ■ existant
│   │   ├── registrations.py                  # ■ existant
│   │   ├── revenue/                          # ★ NOUVEAU
│   │   │   ├── __init__.py
│   │   │   ├── crm.py                       # /api/revenue/crm/*
│   │   │   ├── memberships.py              # /api/revenue/memberships/*
│   │   │   ├── campaigns.py                # /api/revenue/campaigns/*
│   │   │   ├── ota.py                      # /api/revenue/ota/*
│   │   │   ├── analytics.py                # /api/revenue/analytics/*
│   │   │   ├── yield_mgmt.py               # /api/revenue/yield/*
│   │   │   ├── attribution.py              # /api/revenue/attribution/*
│   │   │   └── ...
│   │   └── models/                          # ★ NOUVEAU – modèles SQLAlchemy
│   │       ├── __init__.py
│   │       ├── base.py                     # Base déclarative + mixins
│   │       ├── user.py
│   │       ├── customer.py
│   │       ├── product.py
│   │       ├── reservation.py
│   │       ├── payment.py
│   │       ├── qr_code.py
│   │       ├── channel.py
│   │       ├── campaign.py
│   │       ├── membership.py
│   │       ├── analytics_event.py
│   │       └── audit_log.py
│   ├── schemas/                              # ★ NOUVEAU – Pydantic v2 DTOs
│   ├── services/
│   │   ├── ops_sync.py                      # ★ Pont Ops ↔ Revenue (events)
│   │   ├── yield_engine.py                 # ★ Calcul de prix dynamiques
│   │   ├── attribution.py                   # ★ Multi-touch attribution
│   │   ├── ota_channel_mgr.py              # ★ Sync OTA
│   │   └── ...
│   ├── workers/                             # ★ NOUVEAU – jobs Celery / RQ
│   │   ├── celery_app.py
│   │   ├── sync_ota.py                     # toutes les 5min
│   │   ├── recalc_lifetime_value.py        # quotidien
│   │   ├── compute_rfm_segments.py         # hebdomadaire
│   │   ├── meta_conversion_api.py          # temps réel
│   │   └── alembic/                        # ★ NOUVEAU – migrations PG
│   │       ├── env.py
│   │       └── versions/
│   ├── tests/
│   │   ├── test_iteration29_scanner_fix.py
│   │   ├── test_qr_audit_boarding.py
│   │   ├── revenue/                         # ★ NOUVEAU
│   │   │   ├── test_crm.py
│   │   │   ├── test_yield.py
│   │   │   └── test_attribution.py
│   ├── core/                                # ★ config, security, db, deps
│   │   ├── config.py
│   │   ├── security.py
│   │   ├── db_mongo.py
│   │   ├── db_postgres.py
│   │   ├── dependencies.py
│   │   ├── requirements.txt
│   │   └── .env

```

```

■■■ frontend/                                # React (existant + extension)
■   ■■■ public/
■       ■■■ Manuel_Planning_BBr.pdf
■       ■■■ Manuel_Cantine_BBr.pdf
■       ■■■ BBR_Revenue_Engine_Architecture.pdf
■   ■■■ src/
■       ■■■ pages/                            # ■ existant
■           ■■■ reservations/
■           ■■■ cantine/
■           ■■■ staff/
■           ■■■ revenue/                      # ★ NOUVEAU – pages du Revenue Engine
■               ■■■ DashboardKpis.jsx
■               ■■■ CrmCustomers.jsx
■               ■■■ CrmSegments.jsx
■               ■■■ CampaignsList.jsx
■               ■■■ CampaignDetail.jsx
■               ■■■ OtaChannels.jsx
■               ■■■ OtaCalendar.jsx
■               ■■■ MembershipsList.jsx
■               ■■■ YieldRules.jsx
■               ■■■ Attribution.jsx
■               ■■■ Reports.jsx
■       ■■■ components/
■           ■■■ ui/                          # shadcn/ui (existant)
■           ■■■ charts/                     # ★ NOUVEAU – Recharts/Tremor
■           ■■■ revenue/                   # ★ Composants Revenue Engine
■           ■■■ stores/                   # ★ NOUVEAU – Zustand
■               ■■■ authStore.ts
■               ■■■ revenueStore.ts
■               ■■■ filtersStore.ts
■       ■■■ hooks/
■           ■■■ revenue/                   # ★ React Query hooks
■               ■■■ useCustomers.ts
■               ■■■ useCampaigns.ts
■               ■■■ useKpis.ts
■       ■■■ lib/
■           ■■■ api.ts                     # client axios
■           ■■■ tracking.ts                # ★ pixel + UTM
■       ■■■ App.jsx
■       ■■■ ...
■■■ infra/                                  # ★ NOUVEAU
■   ■■■ docker-compose.yml
■   ■■■ postgres/
■       ■■■ init.sql
■   ■■■ redis/
■   ■■■ cloudflare/
■       ■■■ wrangler.toml
■■■ docs/                                  # ★ NOUVEAU
■   ■■■ ARCHITECTURE.md
■   ■■■ DATA_MODEL.md
■   ■■■ INTEGRATIONS.md
■   ■■■ RUNBOOK.md
■■■ scripts/                              # ■ existant
■   ■■■ generate_planning_manual.py
■   ■■■ generate_cantine_manual.py
■   ■■■ generate_revenue_engine_pdf.py
■   ■■■ revenue_engine_diagrams.py
■■■ memory/
■   ■■■ PRD.md

```

Convention de nommage

■ **existant** = code de l'app actuelle, conservé tel quel. ★ **NOUVEAU** = code à créer dans la phase Revenue Engine. Aucun fichier existant n'est supprimé. Toutes les nouvelles routes sont préfixées `/api/revenue/*` pour faciliter le monitoring, le rate-limiting et le déploiement progressif.

Schéma UML des modules

Le Revenue Engine est structuré en 9 modules fonctionnels. Le module **CRM** est central — toutes les autres briques le référencent. Le module **Marketing** alimente le CRM en leads et conversions. Le module **Analytics** consomme l'ensemble des autres modules pour produire des indicateurs.

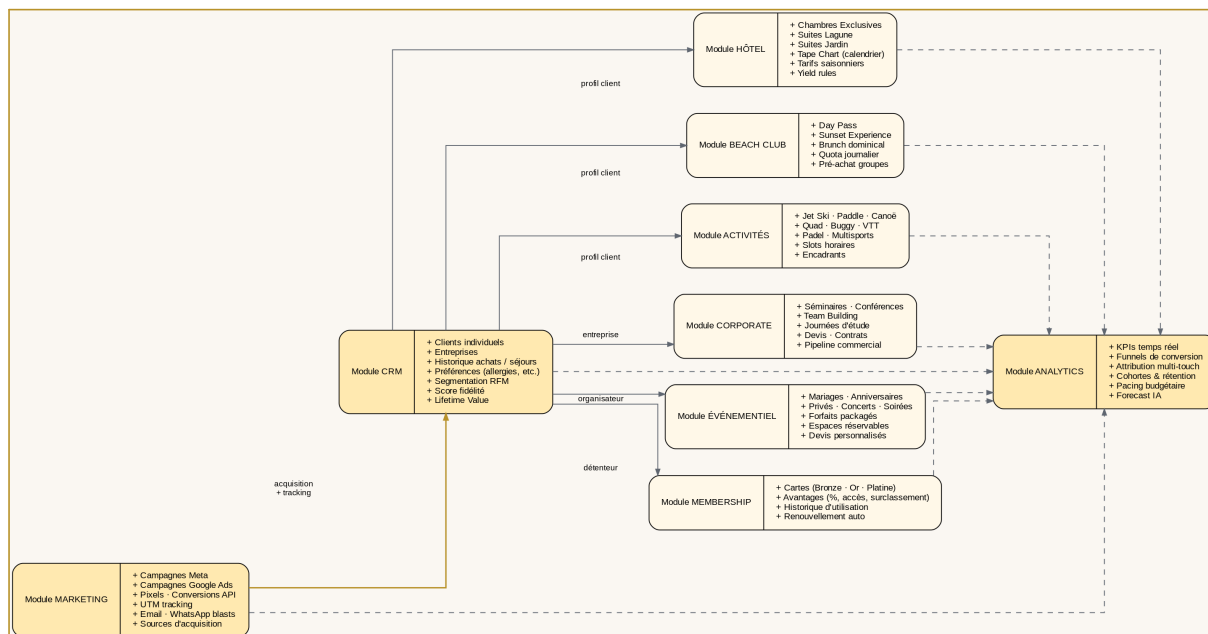


Schéma 2 — Modules fonctionnels et leurs dépendances.

Modules métier — produits réservables

Cinq modules incarnent l'offre commerciale de BBr : **Hôtel**, **Beach Club**, **Activités**, **Corporate** et **Événementiel**. Tous partagent un référentiel commun *Product / Inventory / Reservation*, mais chacun apporte ses règles spécifiques (slots horaires pour les activités, devis pour le corporate, packagings pour l'événementiel, etc.).

Modules transversaux

Quatre modules orchestrent l'expérience client : **CRM** (référentiel client + segmentation), **Membership** (programme de fidélité), **Marketing** (acquisition + retargeting), **Analytics** (pilotage + attribution).

Schéma UML de la base de données

Le schéma relationnel ci-dessous présente les 13 entités principales du Revenue Engine PostgreSQL avec leurs attributs clés et leurs relations.

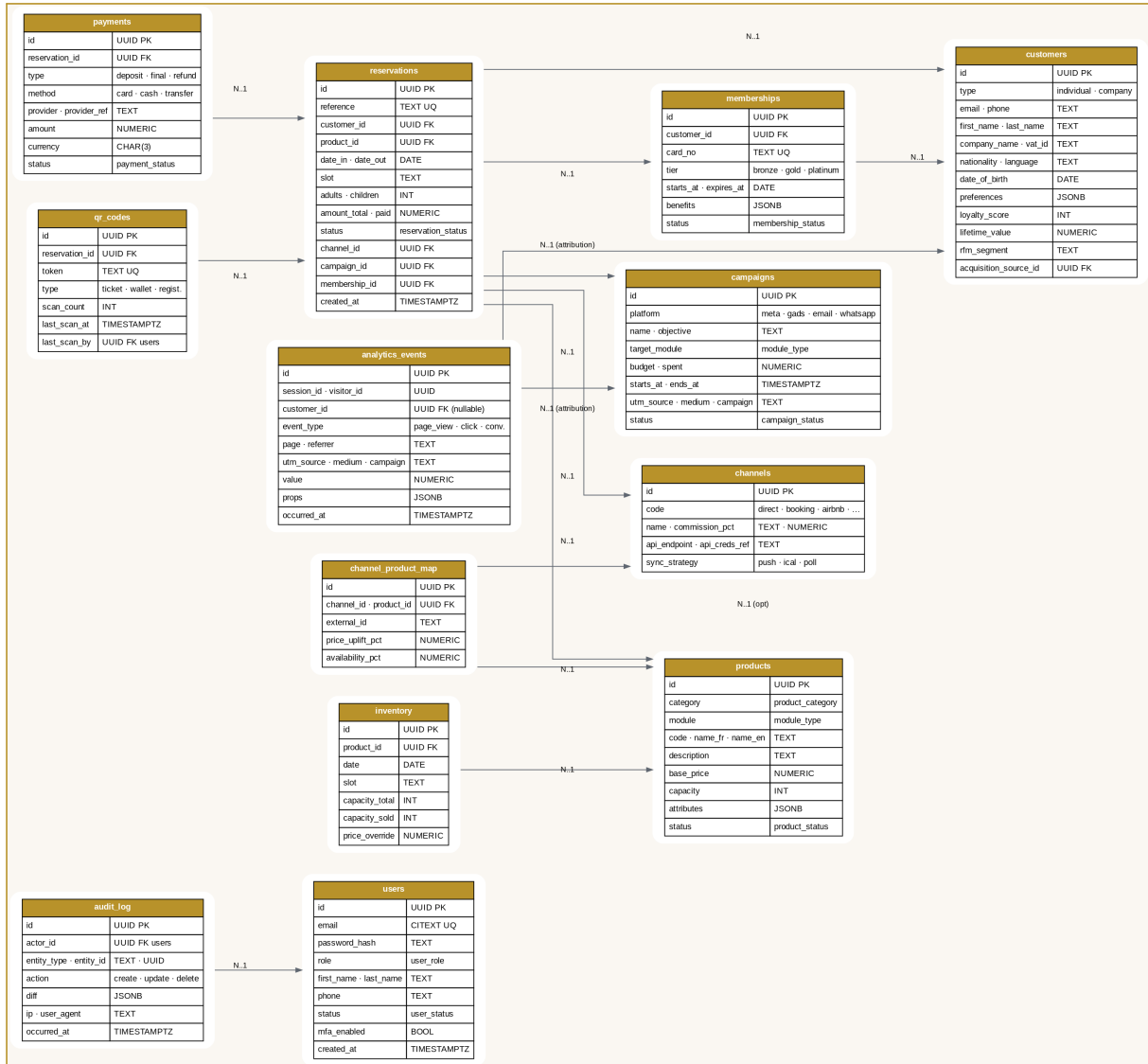


Schéma 3 — ERD complet du schéma PostgreSQL revenue_engine.

Modèle relationnel PostgreSQL (DDL)

Le DDL complet est livré ci-dessous, regroupé en deux blocs : **types énumérés** puis **tables**. Tous les objets vivent dans le schéma `revenue_engine` pour isolation et facilité de backup. Les UUID sont la clé primaire universelle (via `gen_random_uuid()`) pour préparer un éventuel sharding ultérieur.

5.1 — Types énumérés

```
-- Types énumérés (revenue_engine.<type>)
CREATE TYPE user_role AS ENUM (
    'super_admin', 'direction', 'marketing', 'reception',
    'comptabilite', 'commercial', 'controle_embarquement'
);
CREATE TYPE user_status AS ENUM ('active', 'suspended', 'deleted');
CREATE TYPE customer_type AS ENUM ('individual', 'company');
CREATE TYPE module_type AS ENUM (
    'hotel', 'beach_club', 'activities',
    'corporate', 'events', 'membership'
);
CREATE TYPE product_category AS ENUM (
    'room', 'suite_lagoon', 'suite_garden',
    'day_pass', 'sunset', 'brunch',
    'jet_ski', 'paddle', 'canoe', 'quad', 'buggy',
    'padel', 'mtb', 'multisport',
    'seminar', 'conference', 'team_building',
    'wedding', 'birthday', 'concert', 'private_event'
);
CREATE TYPE product_status AS ENUM ('active', 'paused', 'archived');
CREATE TYPE reservation_status AS ENUM (
    'pending', 'confirmed', 'paid', 'checked_in',
    'completed', 'cancelled', 'no_show', 'refunded'
);
CREATE TYPE payment_type AS ENUM ('deposit', 'final', 'refund', 'tip');
CREATE TYPE payment_method AS ENUM (
    'card', 'mobile_money', 'cash', 'transfer', 'voucher'
);
CREATE TYPE payment_status AS ENUM (
    'pending', 'authorized', 'captured', 'failed', 'refunded'
);
CREATE TYPE membership_tier AS ENUM ('bronze', 'gold', 'platinum');
CREATE TYPE membership_status AS ENUM (
    'active', 'expired', 'suspended', 'cancelled'
);
CREATE TYPE campaign_status AS ENUM (
    'draft', 'scheduled', 'active', 'paused', 'completed', 'archived'
);
CREATE TYPE qr_code_type AS ENUM ('ticket', 'wallet', 'registration');
```

5.2 — Tables principales

```
-- Schéma principal
CREATE SCHEMA IF NOT EXISTS revenue_engine;
SET search_path TO revenue_engine, public;

-- 1. UTILISATEURS BACK-OFFICE
CREATE TABLE users (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email             CITEXT UNIQUE NOT NULL,
    password_hash     TEXT NOT NULL,
    role              user_role NOT NULL,
    first_name        TEXT NOT NULL,
    last_name         TEXT NOT NULL,
    phone             TEXT,
    status            user_status NOT NULL DEFAULT 'active',
    mfa_enabled        BOOLEAN NOT NULL DEFAULT false,
    mfa_secret_enc     BYTEA,
    last_login_at     TIMESTAMPTZ,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at        TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_users_role ON users(role) WHERE status = 'active';

-- 2. CLIENTS / ENTREPRISES (CRM)
CREATE TABLE customers (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    type              customer_type NOT NULL,
    email             CITEXT,
    phone             TEXT,
    first_name        TEXT,
    last_name         TEXT,
    company_name      TEXT,
    vat_id            TEXT,
    nationality        CHAR(2),
    language          CHAR(2) DEFAULT 'fr',
    date_of_birth     DATE,
    preferences       JSONB NOT NULL DEFAULT '{}',
    loyalty_score     INTEGER NOT NULL DEFAULT 0,
    lifetime_value    NUMERIC(12,2) NOT NULL DEFAULT 0,
    rfm_segment       TEXT,
    acquisition_source_id UUID REFERENCES campaigns(id) ON DELETE SET NULL,
    consent_marketing BOOLEAN NOT NULL DEFAULT false,
    consent_marketing_at TIMESTAMPTZ,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
    CHECK (
        (type = 'individual' AND first_name IS NOT NULL)
        OR (type = 'company' AND company_name IS NOT NULL)
    )
);
CREATE INDEX idx_customers_email ON customers(email) WHERE email IS NOT NULL;
CREATE INDEX idx_customers_phone ON customers(phone) WHERE phone IS NOT NULL;
CREATE INDEX idx_customers_rfm ON customers(rfm_segment);

-- 3. PRODUITS
CREATE TABLE products (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    module            module_type NOT NULL,
    category          product_category NOT NULL,
    code              TEXT UNIQUE NOT NULL,
    name_fr           TEXT NOT NULL,
    name_en           TEXT,
    description        TEXT,
    base_price        NUMERIC(10,2) NOT NULL,
    currency          CHAR(3) NOT NULL DEFAULT 'XOF',
    capacity          INTEGER,
    duration_min      INTEGER,
    attributes        JSONB NOT NULL DEFAULT '{}',
    status            product_status NOT NULL DEFAULT 'active',
    created_at        TIMESTAMPTZ NOT NULL DEFAULT now()
```

```

);

-- 4. INVENTAIRE (capacité par produit/date/slot)
CREATE TABLE inventory (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  product_id  UUID NOT NULL REFERENCES products(id) ON DELETE CASCADE,
  date        DATE NOT NULL,
  slot        TEXT,                                -- '10H', '16H', null = all-day
  capacity_total INTEGER NOT NULL,
  capacity_sold INTEGER NOT NULL DEFAULT 0,
  price_override NUMERIC(10,2),
  UNIQUE (product_id, date, slot)
);
CREATE INDEX idx_inventory_date ON inventory(date);
CREATE INDEX idx_inventory_avail ON inventory(product_id, date)
  WHERE capacity_sold < capacity_total;

-- 5. CANAUX DE DISTRIBUTION
CREATE TABLE channels (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  code        TEXT UNIQUE NOT NULL,
  name        TEXT NOT NULL,
  commission_pct NUMERIC(5,2) NOT NULL DEFAULT 0,
  api_endpoint TEXT,
  api_creds_ref TEXT,                                -- ref to secrets vault
  sync_strategy TEXT NOT NULL,                      -- 'push', 'ical', 'poll'
  is_active   BOOLEAN NOT NULL DEFAULT true
);

-- 6. MAPPING PRODUITS ↔ CANAUX
CREATE TABLE channel_product_map (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  channel_id  UUID NOT NULL REFERENCES channels(id),
  product_id  UUID NOT NULL REFERENCES products(id),
  external_id TEXT NOT NULL,
  price_uplift_pct NUMERIC(5,2) NOT NULL DEFAULT 0,
  availability_pct NUMERIC(5,2) NOT NULL DEFAULT 100,
  last_sync_at TIMESTAMPTZ,
  UNIQUE (channel_id, product_id)
);

-- 7. CAMPAGNES MARKETING
CREATE TABLE campaigns (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  platform    TEXT NOT NULL,                        -- 'meta', 'gads', 'email', 'whatsapp'
  name        TEXT NOT NULL,
  objective    TEXT,
  target_module module_type,
  budget       NUMERIC(10,2),
  spent        NUMERIC(10,2) NOT NULL DEFAULT 0,
  starts_at    TIMESTAMPTZ,
  ends_at      TIMESTAMPTZ,
  utm_source   TEXT,
  utm_medium   TEXT,
  utm_campaign TEXT,
  external_id  TEXT,                                -- ad set / campaign id côté plateforme
  status       campaign_status NOT NULL DEFAULT 'draft',
  created_by   UUID REFERENCES users(id),
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_campaigns_utm ON campaigns(utm_source, utm_medium, utm_campaign);
CREATE INDEX idx_campaigns_active ON campaigns(status) WHERE status = 'active';

-- 8. MEMBERSHIPS
CREATE TABLE memberships (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  customer_id UUID NOT NULL REFERENCES customers(id) ON DELETE CASCADE,
  card_no     TEXT UNIQUE NOT NULL,
  tier         membership_tier NOT NULL,
  starts_at   DATE NOT NULL,
  expires_at  DATE NOT NULL,
  benefits    JSONB NOT NULL DEFAULT '{}',

```

```

        auto_renew    BOOLEAN NOT NULL DEFAULT true,
        status        membership_status NOT NULL DEFAULT 'active',
        created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
    );
CREATE INDEX idx_memberships_customer ON memberships(customer_id);
CREATE INDEX idx_memberships_active ON memberships(status, expires_at)
    WHERE status = 'active';

-- 9. RÉSERVATIONS
CREATE TABLE reservations (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    reference         TEXT UNIQUE NOT NULL,
    customer_id       UUID NOT NULL REFERENCES customers(id),
    product_id        UUID NOT NULL REFERENCES products(id),
    date_in           DATE NOT NULL,
    date_out          DATE,
    slot              TEXT,
    adults            INTEGER NOT NULL DEFAULT 1,
    children          INTEGER NOT NULL DEFAULT 0,
    amount_total      NUMERIC(12,2) NOT NULL,
    amount_paid       NUMERIC(12,2) NOT NULL DEFAULT 0,
    currency          CHAR(3) NOT NULL DEFAULT 'XOF',
    status            reservation_status NOT NULL DEFAULT 'pending',
    channel_id        UUID REFERENCES channels(id),
    campaign_id       UUID REFERENCES campaigns(id),
    membership_id     UUID REFERENCES memberships(id),
    notes             TEXT,
    metadata          JSONB NOT NULL DEFAULT '{}',
    -- Link back to Operations Mongo document
    ops_booking_id    TEXT,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at        TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_res_customer ON reservations(customer_id);
CREATE INDEX idx_res_product_date ON reservations(product_id, date_in);
CREATE INDEX idx_res_status ON reservations(status);
CREATE INDEX idx_res_channel ON reservations(channel_id);
CREATE INDEX idx_res_campaign ON reservations(campaign_id);

-- 10. PAIEMENTS
CREATE TABLE payments (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    reservation_id    UUID NOT NULL REFERENCES reservations(id) ON DELETE RESTRICT,
    type              payment_type NOT NULL,
    method            payment_method NOT NULL,
    provider          TEXT, -- 'fineopay', 'stripe', 'cash'
    provider_ref      TEXT,
    amount            NUMERIC(12,2) NOT NULL,
    currency          CHAR(3) NOT NULL DEFAULT 'XOF',
    status            payment_status NOT NULL DEFAULT 'pending',
    failure_reason    TEXT,
    metadata          JSONB NOT NULL DEFAULT '{}',
    created_at        TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_pay_res ON payments(reservation_id);
CREATE INDEX idx_pay_status ON payments(status);

-- 11. QR CODES
CREATE TABLE qr_codes (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    reservation_id    UUID NOT NULL REFERENCES reservations(id) ON DELETE CASCADE,
    type              qr_code_type NOT NULL,
    token             TEXT UNIQUE NOT NULL,
    payload_compact   TEXT NOT NULL, -- the JSON encoded in the QR
    scan_count        INTEGER NOT NULL DEFAULT 0,
    last_scan_at      TIMESTAMPTZ,
    last_scan_by      UUID REFERENCES users(id),
    revoked           BOOLEAN NOT NULL DEFAULT false,
    revoked_at        TIMESTAMPTZ,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_qr_token ON qr_codes(token) WHERE NOT revoked;

```

```

-- 12. ÉVÉNEMENTS ANALYTICS (table volumineuse – partitionnée par mois)
CREATE TABLE analytics_events (
    id                UUID NOT NULL DEFAULT gen_random_uuid(),
    session_id        UUID,
    visitor_id         UUID,
    customer_id        UUID REFERENCES customers(id) ON DELETE SET NULL,
    event_type         TEXT NOT NULL,                -- 'page_view', 'add_to_cart', 'purchase', ...
    page               TEXT,
    referrer            TEXT,
    utm_source          TEXT,
    utm_medium          TEXT,
    utm_campaign        TEXT,
    value              NUMERIC(12,2),
    currency            CHAR(3),
    props              JSONB NOT NULL DEFAULT '{}',
    occurred_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (id, occurred_at)
) PARTITION BY RANGE (occurred_at);
-- Partitions mensuelles créées via pg_partman.

-- 13. JOURNAL D'AUDIT (write-once, append-only)
CREATE TABLE audit_log (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    actor_id           UUID REFERENCES users(id),
    entity_type        TEXT NOT NULL,
    entity_id          UUID NOT NULL,
    action             TEXT NOT NULL,                -- 'create', 'update', 'delete'
    diff               JSONB NOT NULL,
    ip                 INET,
    user_agent         TEXT,
    occurred_at         TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_audit_entity ON audit_log(entity_type, entity_id);
CREATE INDEX idx_audit_actor ON audit_log(actor_id, occurred_at DESC);

```

Extensions PostgreSQL requises

uuid-oss (UUID v4) · citext (emails case-insensitive) · pg_trgm (recherche approximative pour le CRM) · pg_partman (partitions analytics_events) · pgcrypto (chiffrement champs sensibles) · btree_gist (exclusion constraints sur inventory).

Tables détaillées

Description fonctionnelle des 13 tables principales. Chaque fiche précise la **finalité**, la **volumétrie attendue** et le **détail des colonnes**.

users

Utilisateurs back-office du Revenue Engine (et du staff opérationnel).

Volumétrie attendue : ≤ 100 lignes — table à très faible cardinalité, mais lue à chaque requête (authentification JWT).

Colonne	Type	Description
id	UUID	Identifiant universel (PK).
email	CITEXT	Email unique, case-insensitive.
password_hash	TEXT	Bcrypt (cost 12) ou Argon2id.
role	user_role	Rôle parmi les 7 rôles définis.
status	user_status	active · suspended · deleted.
mfa_enabled / mfa_secret_enc	BOOL/BYTEA	TOTP optionnel, secret chiffré.
last_login_at	TIMESTAMPTZ	Pour détecter les comptes dormants.

customers

Référentiel central CRM — individus et entreprises. Dédupliqué sur (email, phone).

Volumétrie attendue : $\approx 500 \rightarrow 50\,000$ lignes sur 3 ans. Croissance $\sim 3\,000$ /mois en régime stabilisé.

Colonne	Type	Description
type	customer_type	individual ou company.
email/phone	CITEXT/TEXT	Clés de déduplication.
company_name/vat_id	TEXT	Pour les comptes B2B.
preferences	JSONB	Allergies, exigences, préférences chambre.
loyalty_score	INT	0–1000, calculé hebdo.
lifetime_value	NUMERIC	Cumul payé sur la vie du client.
rfm_segment	TEXT	Champion · Loyal · Risk · Lost · ...
acquisition_source_id	FK campaigns	Première campagne ayant converti.
consent_marketing	BOOL	Conformité RGPD/UE.

products

Catalogue des produits réservables, tous modules confondus.

Volumétrie attendue : $\approx 100\text{--}200$ lignes. Croissance lente.

Colonne	Type	Description
module	module_type	hotel, beach_club, activities, ...
category	product_category	room, day_pass, jet_ski, wedding, ...

Colonne	Type	Description
code	TEXT	Code court unique (ex. DAYPASS_ADULT).
base_price/currency	NUMERIC/CHAR(3)	Prix de référence.
capacity	INT	Quantité max par slot (nullable).
duration_min	INT	Durée du service (jet ski 30 min, etc.).
attributes	JSONB	Spécifs libres : age min, équipement fourni.

inventory

Capacité disponible par **produit × date × slot**. Table clé du yield management.

Volumétrie attendue : $\approx 365 \times 100 \text{ produits} \times 3 \text{ slots} = \sim 100\,000 \text{ lignes/an}$.

Colonne	Type	Description
product_id + date + slot	UNIQUE	Triplet métier.
capacity_total	INT	Capacité brute du créneau.
capacity_sold	INT	Compteur mis à jour à chaque booking.
price_override	NUMERIC	Surcharge prix (yield ou promo).

channels & channel_product_map

Canaux de distribution OTA + mapping fin par produit (prix uplift, % d'allotement, ID externe).

Volumétrie attendue : $\text{channels} \leq 20 \cdot \text{mapping} \approx 20 \times 100 = 2\,000 \text{ lignes}$.

Colonne	Type	Description
code	TEXT	direct · booking · airbnb · expedia · ...
commission_pct	NUMERIC	0 pour direct, 18% Booking, ...
sync_strategy	TEXT	push API · ical · poll.
external_id	TEXT	ID hôtel/listing sur la plateforme.
price_uplift_pct	NUMERIC	Marge OTA appliquée.
availability_pct	NUMERIC	% du stock alloué au canal.

campaigns

Campagnes Marketing (Meta, Google Ads, Email, WhatsApp). UTM unifié.

Volumétrie attendue : $\approx 50\text{--}200 \text{ campagnes actives à tout moment}$.

Colonne	Type	Description
platform	TEXT	meta · gads · email · whatsapp · referral.
target_module	module_type	Module ciblé.
budget/spent	NUMERIC	Budget alloué vs consommé (pacing).
utm_source/medium/campaign	TEXT	Pour l'attribution.
external_id	TEXT	ID côté plateforme.
status	campaign_status	draft · scheduled · active · paused · ...

memberships

Cartes de fidélité — Bronze / Or / Platine. Auto-renouvellement annuel.

Volumétrie attendue : $\approx 1\,000 \rightarrow 10\,000$ cartes actives.

Colonne	Type	Description
card_no	TEXT UNIQUE	Numéro lisible (ex. BBR-2026-0042).
tier	membership_tier	bronze · gold · platinum.
starts_at / expires_at	DATE	Validité de 1 an.
benefits	JSONB	% réduc, accès, surclassement, plages, ...
auto_renew	BOOL	Renouvellement automatique.

reservations

Cœur transactionnel — toute vente est une réservation, quel que soit le module. Référence visible BBR-AB12-XY34.

Volumétrie attendue : $\approx 10\,000$ / mois en régime de croisière. Plus de 1 M sur 5 ans.

Colonne	Type	Description
reference	TEXT UNIQUE	Ref humaine pour l'agent et le client.
customer_id / product_id	FK	Qui a réservé quoi.
date_in / date_out / slot	DATE/TEXT	Quand.
amount_total / amount_paid	NUMERIC	Encaissé vs total dû.
status	reservation_status	8 états du tunnel (pending → completed).
channel_id / campaign_id / membership_id	FK	Attribution complète.
ops_booking_id	TEXT	Pont vers le document MongoDB Ops.
metadata	JSONB	Champs libres (notes, options, source).

payments

Mouvements financiers (acomptes, soldes, remboursements). Une réservation peut avoir 1..N paiements.

Volumétrie attendue : $\approx 1,2 \times$ reservations $\approx 12\,000$ /mois.

Colonne	Type	Description
reservation_id	FK	Réservation rattachée.
type	payment_type	deposit · final · refund · tip.
method	payment_method	card · mobile_money · cash · transfer.
provider/provider_ref	TEXT	Référence du PSP (FineoPay, Stripe).
amount/currency	NUMERIC/CHAR(3)	Montant + devise.
status	payment_status	pending · authorized · captured · failed · refunded.

qr_codes

Tickets et passes (boarding pass, wallet, registration). Chaque réservation génère 1..N QR.

Volumétrie attendue : $\approx 1,5 \times$ reservations $\approx 15\,000$ /mois.

Colonne	Type	Description
token	TEXT UNIQUE	32-hex random (cryptographiquement sûr).
type	qr_code_type	ticket · wallet · registration.

Colonne	Type	Description
payload_compact	TEXT	JSON ~75 chars stocké dans le QR.
scan_count / last_scan_at / last_scan_by		Audit du scan.
revoked	BOOL	Désactivation manuelle (perte, fraude).

analytics_events

Flux temps réel d'événements (page views, clics, conversions). Partitionné mensuellement pour des purges efficaces.

Volumétrie attendue : ≈ 1 M lignes / mois. Conservation 13 mois en hot, archive S3 au-delà.

Colonne	Type	Description
session_id / visitor_id	UUID	Anonyme avant login.
customer_id	FK (nullable)	Une fois identifié.
event_type	TEXT	page_view · add_to_cart · purchase · ...
utm_source/medium/campaign	TEXT	Capture l'origine.
value	NUMERIC	Montant si conversion.
props	JSONB	Données contextuelles (device, page, etc.).
occurred_at	TIMESTAMPTZ	Clé de partitionnement.

audit_log

Journal append-only des actions critiques (write tables). Pour la conformité, la traçabilité et le forensic.

Volumétrie attendue : ≈ 50 000 lignes/mois. Rotation S3 trimestrielle.

Colonne	Type	Description
actor_id	FK users	Qui a fait l'action.
entity_type / entity_id	TEXT/UUID	Sur quoi.
action	TEXT	create · update · delete · login · ...
diff	JSONB	Avant/après pour les updates.
ip / user_agent	INET/TEXT	Forensic réseau.

Relations entre tables

Toutes les relations utilisent les clés étrangères UUID avec règle de suppression explicite. **RESTRICT** empêche la suppression d'une entité parente liée à des enfants. **CASCADE** supprime les enfants en cascade. **SET NULL** préserve l'historique tout en libérant la référence.

Table parente	Table enfant	FK	Cardinalité	Suppression
customers	reservations	customer_id	1..N	RESTRICT
customers	memberships	customer_id	1..N	CASCADE
customers	analytics_events	customer_id	1..N	SET NULL
products	inventory	product_id	1..N	CASCADE
products	reservations	product_id	1..N	RESTRICT
products	channel_product_map	product_id	1..N	CASCADE
reservations	payments	reservation_id	1..N	RESTRICT
reservations	qr_codes	reservation_id	1..N	CASCADE
channels	channel_product_map	channel_id	1..N	CASCADE
channels	reservations	channel_id	1..N	SET NULL
campaigns	reservations	campaign_id	1..N	SET NULL
campaigns	customers	acquisition_source_id	1..N	SET NULL
campaigns	analytics_events	(via utm_*)	1..N	n/a
memberships	reservations	membership_id	1..N	SET NULL
users	qr_codes	last_scan_by	1..N	SET NULL
users	audit_log	actor_id	1..N	SET NULL
users	campaigns	created_by	1..N	SET NULL

Pourquoi RESTRICT sur customers ↔ reservations ?

Un client avec un historique de réservations ne doit jamais être supprimé physiquement — il est **anonymisé** (RGPD) via une procédure dédiée qui efface les PII tout en conservant l'historique commercial pour l'analyse.

Contraintes complémentaires

- **Exclusion constraint** sur inventory(product_id, date, slot) via btree_gist — empêche deux lignes pour le même triplet.
- **Check constraint** amount_paid <= amount_total + tolerance sur reservations — sécurité comptable.
- **Check constraint** sur customers : individu OU entreprise (jamais les deux, jamais aucun).
- **Trigger d'audit** sur les tables sensibles (users, customers, reservations, payments) — chaque UPDATE écrit automatiquement dans audit_log.

Politique de sécurité

La sécurité du Revenue Engine s'appuie sur une approche **défense en profondeur** à 6 niveaux. Chaque niveau indépendant garantit que la compromission d'un seul ne suffit pas à mettre l'ensemble en péril.

8.1 — Matrice de rôles (RBAC)

Sept rôles fonctionnels, chacun avec un périmètre de droits explicite. Le principe est **moindre privilège** systématique.

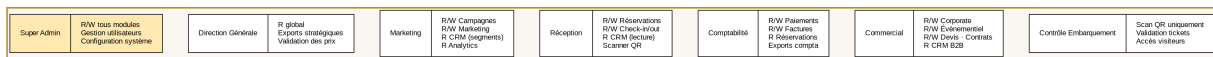


Schéma 4 — Vue d'ensemble des 7 rôles utilisateurs.

Module	Super Admin	Direction	Marketing	Réception	Compta	Commercial	Embarq.
Hôtel	R/W	R	R	R/W	R	R	—
Beach Club	R/W	R	R	R/W	R	R	—
Activités	R/W	R	R	R/W	R	R	—
Corporate	R/W	R	R	R	R	R/W	—
Événementiel	R/W	R	R	R	R	R/W	—
Membership	R/W	R	R/W	R	R	R/W	—
CRM	R/W	R	R/W	R	R	R/W	—
Marketing	R/W	R	R/W	—	—	R	—
Analytics	R/W	R	R	R	R	R	—
Paiements	R/W	R	—	R	R/W	R	—
QR Scan	R/W	—	—	R/W	—	—	R/W
Utilisateurs	R/W	R	—	—	—	—	—
Config système	R/W	—	—	—	—	—	—

8.2 — Row-Level Security (équivalent PostgreSQL)

Approche analogue au RLS Supabase, native dans PostgreSQL depuis 9.5. Chaque table sensible expose une policy qui filtre les lignes en fonction du rôle de la session (`current_setting('app.user_role')`) et de l'identité (`current_setting('app.user_id')`) positionnées par le middleware FastAPI au début de chaque requête.

```
-- Exemple sur la table reservations
ALTER TABLE reservations ENABLE ROW LEVEL SECURITY;

CREATE POLICY reservations_read ON reservations
FOR SELECT USING (
  current_setting('app.user_role')::text IN
    ('super_admin', 'direction', 'reception',
     'comptabilite', 'marketing')
OR (
  current_setting('app.user_role') = 'commercial'
  AND EXISTS (SELECT 1 FROM products p
               WHERE p.id = product_id
```

```

        AND p.module IN ('corporate', 'events'))
    )
);

CREATE POLICY reservations_write ON reservations
FOR ALL USING (
    current_setting('app.user_role') IN ('super_admin', 'reception')
);

```

8.3 — Journal d'activité (audit_log)

Toutes les opérations *write* sensibles laissent une trace dans `audit_log` via des triggers PG. Le journal est append-only (CREATE/DELETE révoqués au rôle applicatif). Les données sont déversées trimestriellement vers S3 Glacier pour archive 7 ans (norme comptable).

8.4 — Historique des modifications

Le champ diff JSONB de `audit_log` stocke un patch *before/after* de chaque mise à jour. L'interface admin permet de rejouer une entité ligne par ligne (forensic et conformité).

8.5 — Protections applicatives

- **CSRF** — token double-submit cookie sur tous les POST/PUT/DELETE.
- **XSS** — Content Security Policy stricte (default-src 'self'), sanitisation systématique des inputs côté FastAPI (Pydantic + bleach).
- **Rate limiting** — Redis-backed (slowapi côté FastAPI + Cloudflare Rate Limiting rules). Limites par IP, par session et par endpoint sensible.
- **JWT** — courte durée (15 min access + 7j refresh rotatif), signés ES256, stockés en HTTP-only Secure SameSite=strict cookies.
- **MFA** — TOTP optionnel pour les rôles sensibles (Super Admin, Direction, Comptabilité), via Authy/Google Authenticator.
- **Chiffrement au repos** — colonnes sensibles (`mfa_secret`, identifiants API canaux) chiffrées avec pgcrypto + clé en HashiCorp Vault.

8.6 — Sauvegardes automatiques

- **WAL streaming** en continu vers un standby PG (RPO < 1 min, RTO < 5 min).
- **Snapshot quotidien** chiffré (AES-256) vers S3, rétention 30 jours hot + 1 an warm + 7 ans glacier.
- **Drill mensuel** de restauration sur environnement isolé — toute sauvegarde non testée est une sauvegarde morte.
- **MongoDB Ops** — mongodump quotidien + Atlas backup continu (déjà en place).

Stratégie de scalabilité

La plateforme est conçue pour absorber une croissance $\times 10$ sur 24 mois **sans refonte architecturale**. Les leviers sont activés progressivement, par paliers de trafic.

9.1 — Palier 1 (lancement → 50 k réservations/an)

- Architecture mono-instance suffit. PG + Mongo + Redis sur 1 VPS dédié 8 vCPU / 16 GB.
- FastAPI servi par uvicorn + gunicorn, 4 workers.
- Frontend statique sur Cloudflare Pages (CDN gratuit).

9.2 — Palier 2 (50 k → 200 k réservations/an)

- **Read replica PostgreSQL** — toutes les requêtes analytics et BI vont sur le réplica, l'écriture reste sur le master.
- **Pgbouncer** en pool de connexions devant PG.
- **Cache Redis** agressif sur les routes catalogue + inventory (TTL 30-60s, invalidation event-driven).
- **Workers asynchrones** (Celery) pour les jobs longs : sync OTA, Meta CAPI, recalcul LTV, export PDF/Excel.
- **Auto-scaling** horizontal des workers FastAPI via Docker Swarm ou Kubernetes léger (k3s).

9.3 — Palier 3 (200 k → 1 M réservations/an)

- **Partitionnement** de reservations et payments par année (déjà partitionné pour analytics_events).
- **ClickHouse** ou **DuckDB** pour les rapports analytics lourds (cubes OLAP), alimenté par CDC depuis PG.
- **Multi-région** — réplique PG en France + Côte d'Ivoire pour réduire la latence d'écriture des bookings OTA internationaux.
- **Sharding** potentiel sur la table analytics_events par visitor_id si volume > 100 M lignes/mois.

9.4 — Observabilité

Pas de scalabilité sans observabilité :

- **Metrics** — Prometheus + Grafana (CPU, RAM, query latency, p95 API).
- **Traces** — OpenTelemetry, ingest vers Jaeger ou Honeycomb (route → service → DB).
- **Logs** — agrégation Loki ou Vector, alerting sur patterns critiques.
- **Uptime monitoring** — Better Uptime / Uptime Kuma, checks toutes les 30s sur les endpoints critiques (booking, paiement, scan QR).

9.5 — Séquence de booking — point critique

Le diagramme ci-dessous illustre le flux complet d'une réservation, du clic sur une pub Meta jusqu'au check-in : 14 étapes orchestrées entre 3 systèmes (front, Ops, Revenue Engine) et 4 services externes

(paiement, OTA, Meta, messaging).

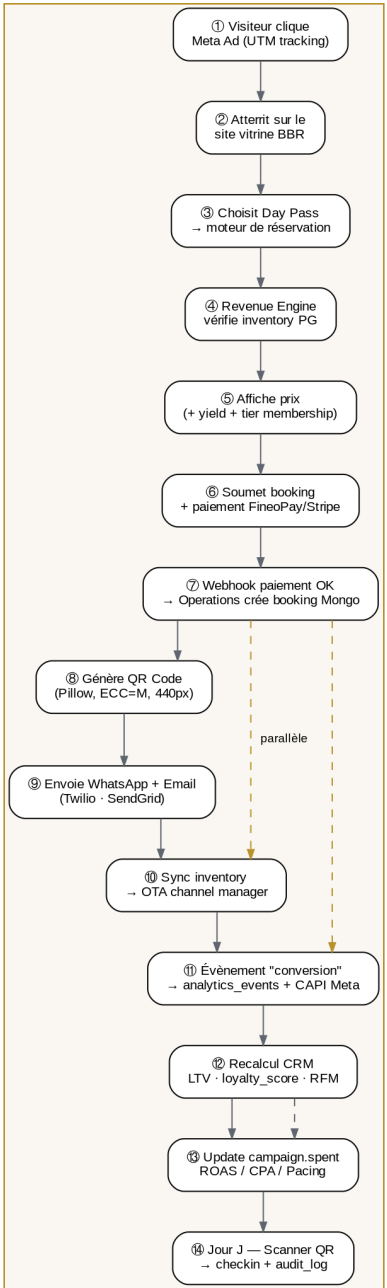


Schéma 5 — Parcours complet d'une réservation, de l'ad au check-in.

Roadmap de développement des modules

Découpage en 7 phases successives livrables tous les 4 à 6 semaines. Chaque phase est **autonome** et **livrable en production** sans dépendance bloquante avec la suivante. Durée totale : **32 semaines** (~8 mois) à partir du kickoff.

Phase	Durée	Modules livrés	Sprint clés
P0 — Fondations	4 semaines	PG schéma · Auth · Audit · RBAC · Pont DB ↔ API	On-premise Auth JWT + MFA · Trigger audit · Webhook
P1 — CRM + Memberships	4 semaines	CRM 360 · Segmentation RFM · Cartes Bronze/Gold/Platine	Profil client enrichi · Score loyalty · Vue 360 · Workflow renou
P2 — Marketing + Analytics	6 semaines	Campagnes Meta + Gads · Pixels · Funnel KPIs · CAPS réel	Pixels KPs · CAPS réel · UTM unifié · Dashboard Tremor · P
P3 — OTA Channel Manager	4 semaines	Booking · Airbnb · Expedia · iCal · Parité d'inventaire	Canaux OTA API · Sync inventory · Mapping produits · Test
P4 — Yield Management	4 semaines	Pricing dynamique · Règles métier · Forecasting	Engine de règles · ML prévision occupation · A/B testing pri
P5 — Corporate + Events	5 semaines	Pipeline commercial · Devis · Contrats · Espaces	Spéc pipeline · Templates devis · DocuSign-like · Calend
P6 — Attribution avancée	3 semaines	Multi-touch attribution · Markov chain · ROM · Clean	Attribution MTA · Reports investisseurs · Forecast revenu IA

Stratégie de livraison

Chaque phase comprend : **conception** → **développement** → **tests automatisés** → **recette utilisateurs** → **mise en production** → **monitoring 1 semaine**. Aucune phase n'est démarrée tant que la précédente n'est pas stable en production (KPI d'incidents < 1/semaine).

Backlog post-MVP

Backlog (post-MVP)	Description
App mobile native	iOS + Android (React Native) pour les concierges et la réception.
IA Concierge	Chatbot WhatsApp pour la réservation conversationnelle.
Loyalty crypto-tokens	Tokenisation des points fidélité sur blockchain (optionnel).
B2B Marketplace	Plateforme de revente aux tour-opérateurs.
Voice search	Réservation par commande vocale sur le site vitrine.
Dynamic content	Personnalisation du site selon le segment du visiteur.

Jalons stratégiques

- **Semaine 4** — Schema PG en production, premier événement écrit dans audit_log.
- **Semaine 12** — Premier campaign tracking end-to-end (Meta ad → conversion → CRM enrichi).
- **Semaine 18** — Première vente via OTA synchronisée automatiquement.
- **Semaine 24** — Prix dynamiques actifs en production sur les modules Hôtel et Day Pass.
- **Semaine 32** — Plateforme complète, attribution multi-touch active, dashboard Direction prêt pour le board.

Conventions et glossaire

Conventions de code

- **Python** — PEP 8, type hints obligatoires, ruff + mypy, docstrings Google style.
- **TypeScript** — strict mode, ESLint Airbnb + Prettier, pas de any en production.
- **SQL** — snake_case, indexes nommés idx_table_columns, FK fk_table_column.
- **Git** — Conventional Commits, branches feature/*, fix/*, chore/*, PRs obligatoires (pas de push direct main).
- **Tests** — pytest pour le backend, Vitest pour le frontend, Playwright pour l'E2E. Coverage minimum 70%.

Conventions API

- Toutes les routes Revenue Engine sont préfixées /api/revenue/{module}.
- Verbes REST stricts (GET / POST / PATCH / DELETE), PATCH pour les updates partiels.
- Pagination cursor-based (pas d'offset/limit sur les tables volumineuses).
- Errors RFC 7807 (Problem Details for HTTP APIs).
- Idempotency keys obligatoires sur les POST de réservation et de paiement (header Idempotency-Key).

Glossaire

Terme	Définition
RFM	Recency · Frequency · Monetary — segmentation client.
LTV	Lifetime Value — somme attendue d'un client sur sa vie.
ROAS	Return on Ad Spend — CA / dépense publicitaire.
CPA	Cost Per Acquisition — coût d'acquisition client.
Yield	Optimisation des prix selon la demande prévue.
OTA	Online Travel Agency — Booking, Airbnb, Expedia, etc.
CAPI	Conversion API — endpoint server-to-server de Meta.
MTA	Multi-Touch Attribution — modèle d'attribution multi-canaux.
RLS	Row-Level Security — filtrage des lignes par utilisateur.
WAL	Write-Ahead Log — journal de transactions PG.
CDC	Change Data Capture — propagation de changements en flux.
RPO	Recovery Point Objective — perte de données max tolérée.
RTO	Recovery Time Objective — temps de remise en service.

Fin du document

Ce document est **vivant**. Il sera versionné dans /app/docs/ARCHITECTURE.md et republié à chaque phase majeure du projet. Les diagrammes sont générés via scripts/revenue_engine_diagrams.py et le PDF via scripts/generate_revenue_engine_pdf.py.